Evaluating the Impact of Agent Harness Components on the Reliability and Efficiency of LLM Agents

Основная идея моего исследования: измерить насколько компоненты в agent harness (мультиагентной системе) влияют на общую эффективность системы.

Кроме этого я хочу предоставить показательные графики, которые позволяют вовремя заметить деградацию системы и переосмыслить добавленный компонент. Некоторые agent harness модульные - это позволит протестировать влияние каждой широко используемой идеи (скиллы, инструменты, инструкции, mcp, lsp)

Предметом исследования выступят известные открытые agent harnesses: pi, opencode, hermes, openhands. Также в рамках исследования планируется разработать свой agent harness на языке go, чтобы иметь полный контроль над экспериментом и тестировать свои гипотезы. После - можно будет сравнивать самописный agent harness с открытыми и широко используемыми

Кроме backend harness будет реализован web UI для отслеживания состояния системы, включая в себя информацию о подключенных mcp, lsp, tools, skills, instructions, etc. И самое главное веб-интерфейс будет содержать данные прогонов на testsuites и понятные метрики, позволяющие отследить развитие системы.

Статья 1

В статье предлагается benchmark (evaluation protocol), у которого зафиксированы начальные условия, то есть для всех прогонов будет одинаковое:

- состояние окружения
- ограничение в токенах
- ограничение во времени
- скрипт валидатор
- условие задачи

В исследовании тестовый набор, кроме сбора обычных метрик, направлен на измерение надежности мульти-агентной системы, так как вводится булева переменная обхода правил безопасности (Security)

Completion вычисляется с помощью task-specific deterministic validators или rubric-based judgment

Process строится из воспроизводимого трейса системы. Он измеряет ошибки окружения, релевантное использование инструмента и логичность, последовательность действий в течении всего цикла.

$$\mathrm{TaskScore}_i = \mathrm{Security}_i \cdot \mathrm{Completion}_i \cdot \mathrm{Process}_i,$$

where $\mathrm{Security}_i \in \{0, 1\}$ and

$$\mathrm{Process}_i = \frac{\mathrm{Robustness}_i + \mathrm{ToolUse}_i + \mathrm{Consistency}_i}{3}.$$

Автор в этом исследовании собрал таблицу типичных симптомов, из-за которых выполнение задачи может быть провалено: the output contract, the tool interface (errors particularly), the evidence base (incomplete usage of given sources), lack of committed artifacts, or continuation state

Статья 2
Задачи в тестовом наборе этого исследования ранжируются по:

- сложности (времени выполнения)
- и по области тестирования, к примеру software engineering, media & content production, office, natural science, cybersec

Эксперименты проводятся на разных agent harnesses. Исследовали эффективность без skills и с подобранными под задачу навыками.

В исследовании представлены результаты, на которые можно равняться и использовать в качестве метрик качества.
Находки:

- Навыки сгенерированные ИИ моделью перед решением задач значительно уступают в эффективности навыкам, которые были тщательно подобраны человеком
- Компактные скиллы, направленные на решение узкой задачи эффективнее скилла, который пытается уместить в себе решение нескольких задач
- Частота вызова скиллов не гарантирует высокое качество решения
- Разные области экспериментов показывают насколько скиллы могут влиять на результат

Статья 3

Эта работа примечательна тем, что тут используются открытые модели не больше 70B параметров каждая. И используются на реальных задачах, которые сейчас выполняют люди. К примеру Code, Web, Game. К тому же в исследовании сразу указаны полезные артефакты https://github.com/THUDM/AgentBench.

Однако тут исследуется не agent harness, а LLM как агент. То есть LLM тренируют выполнять конкретную задачу. Тренировка LLM как агента на одной узкой задаче, в свою очередь, плохо сказывается на overall performance в общих задачах.

Исследование в очередной раз выявило, что качество данных тренировки моделей сильно влияет на конечный performance. Модель, которая училась на данных подкрепленных реальными людьми находится на уровне модели, размер которой превышает её в 3 раза.

Изучив какие статьи есть на интересующую меня тему, я уверен, что эта перспективная и актуальная тема для исследования, так как все статьи написаны в течении последних 5 лет и имеют смежные с моей темой вопросы.

P.S.

Неорганизованные идеи для экспериментов в рамках своего agent harness:

- Сделать систему которая сама себе улучшает и будет знать какая часть её улучшила а какая ухудшила (иметь метрики), чтобы в автономном режиме действительно улучшать себя

- Добавить концепцию статичных хуков (проверок (линт, тест, билд), скриптов, вызовов тулов (сжатие контекста, сохранение основных инсайтов в персистентную векторизированную память)) по триггеру (к примеру завершение задачи или завершение ответа одного из агентов)
- Сохранение основных инсайтов в персистентную векторизированную память -> затем представлять в виде графа, где будут видны связи между заученными правилами
- Сделать поле в поле ввода, тип которого можно выбрать в выпадающем списке или указать через текст (md). У разных типов будут статичные правила обработки текста, что позволит экономить контекст и передавать ИИ только релевантную информацию. К примеру есть логи в которых есть информация о версиях библиотек, которые устарели. Скрипт обработает raw logs в список версий которые нужно обработать и это будет передано агенту, а не 200 строк с ненужными логами
- Для такой системы необходим аудит изменений конфигурационных файлов и отдельных компонентов.
- Закрепленное за пользователем право выбрать тип работы агентов из:
    - Mixture of Agents (есть оркестр агентов)
    - Multi-Agent Debate (несколько агентов делают одну и ту же задачу, потом решают и принимают общее решение)
    - Reflection (агент делает задачу и получает фидбек -> loop)
    - Code Execution (исполнение кода)
    - Concurrent Agents (параллельные агенты)
    - Sequential Workflow (последовательное выполнение подзадач)
    - Handoffs (Один агент делегирует задачи другим)
- Изучение метрик существующих харнесс систем может выявить их недостатки и предложить решение проблем, за счет чего будет возможность повысить их эффективность

  К примеру идея с определением сложности задачи. Часто бывает что из-за перегруза контекста инструментами, mcp, etc. на элементарный вопрос модель тратит 10-ки тысяч токенов. Можно на основе сложности задачи, включать/отключать определенные части системы. И проверить эту гипотезу поможет предлагаемая мной система
- Можно сделать свой набор актуальных тестов, задачи, которые релевантны для меня, как для студента
- Идея создания более простых задач, для использования более простых моделей, что позволит более точечно и явно отслеживать изменение в качестве решения.